

Université / École d'Ingénierie
Département Informatique & Intelligence Artificielle

Rapport Technique – Projet de Machine Learning & Deep Learning

Classification des Battements Cardiaques (ECG Heartbeat)

Compétition Intra-Promotion – Dataset Kaggle

Auteur : [Votre Nom Prénom]

Filière : [Votre filière]

Année : 2025–2026

Encadrant : [Nom de l'enseignant]

4 mai 2026

0 Table des matières

1 Introduction

1.1 Contexte et Problématique

Les maladies cardiovasculaires représentent la première cause de mortalité mondiale selon l’Organisation Mondiale de la Santé. L’électrocardiogramme (ECG) est l’outil diagnostique de référence pour détecter les arythmies cardiaques. Cependant, l’interprétation manuelle des signaux ECG est longue, subjective et requiert une expertise clinique avancée.

L’automatisation de cette classification par des algorithmes de Machine Learning et de Deep Learning constitue donc un enjeu médical majeur, permettant un diagnostic plus rapide, plus objectif et potentiellement plus accessible.

1.2 Objectifs du Projet

Ce projet s’inscrit dans le cadre d’une compétition intra-promotion inspirée des compétitions Kaggle. Les objectifs pédagogiques sont :

- Maîtriser le pipeline complet d’un projet de data science
- Appliquer et comparer des algorithmes ML classiques, ANN, RNN et CNN
- Développer l’esprit critique dans l’analyse et l’interprétation des résultats
- S’initier à la comparaison objective de modèles via des métriques standardisées
- Produire un rapport technique clair et structuré

1.3 Description du Dataset

Le jeu de données utilisé est le *ECG Heartbeat Categorization Dataset* disponible sur Kaggle [?]. Il s’agit de signaux ECG issus de la base de données MIT-BIH Arrhythmia, préalablement segmentés et prétraités.

TABLE 1 – Caractéristiques du dataset MIT-BIH

Ensemble	Échantillons	Features
Train	87 554	187
Test	21 892	187

Chaque signal est représenté par 187 points temporels normalisés dans l’intervalle $[0, 1]$. La variable cible est la classe du battement :

TABLE 2 – Distribution des classes dans le jeu d’entraînement

Classe	Type	Effectif	Proportion
0	Normal	72 471	82,77%
1	Supraventriculaire	2 223	2,54%
2	Ventriculaire	5 788	6,61%
3	Fusion	641	0,73%
4	Inconnu	6 431	7,35%
Total		87 554	100%

2 Analyse Exploratoire des Données (EDA)

2.1 Statistiques Descriptives

L’analyse des statistiques descriptives révèle que les données sont déjà proprement normalisées :

TABLE 3 – Statistiques globales du dataset d’entraînement

Statistique	Valeur
Valeur minimale	0,0000
Valeur maximale	1,0000
Moyenne globale	0,1743
Écart-type	0,2263
Valeurs manquantes	0
Valeurs infinies	0

Les données sont complètes, sans valeurs aberrantes ni manquantes, ce qui confirme la qualité du prétraitement effectué en amont par les auteurs du dataset.

2.2 Déséquilibre des Classes

L’analyse de la distribution révèle un **déséquilibre important** : la classe *Normal* représente à elle seule 82,77% du dataset. Ce déséquilibre est une caractéristique inhérente aux données cliniques réelles, où les battements normaux sont naturellement majoritaires. Il constitue cependant le principal défi de modélisation, car les algorithmes tendent à favoriser la classe majoritaire.

Les classes les plus minoritaires – *Fusion* (0,73%) et *Supraventriculaire* (2,54%) – sont celles pour lesquelles on attend les performances les plus faibles.

2.3 Visualisation des Signaux ECG

L’analyse visuelle des signaux moyens par classe révèle des morphologies distinctes :

- **Normal (Classe 0)** : Pic QRS net et étroit, suivi d’un signal calme. Morphologie la plus reconnaissable.
- **Supraventriculaire (Classe 1)** : Forme plus étalée, pic moins prononcé, complexe QRS diffus.
- **Ventriculaire (Classe 2)** : Pic large et ample, caractéristique d’une dépolarisation ventriculaire anormale.
- **Fusion (Classe 3)** : Signal mixte présentant une double bosse, résultat d’une fusion entre un battement normal et ventriculaire.
- **Inconnu (Classe 4)** : Forme atypique avec un plateau central, morphologie non classifiable dans les catégories standard.

Ces différences morphologiques justifient l’utilisation de modèles capables de capturer des **patterns locaux** dans le signal temporel.

3 Préparation et Prétraitement des Données

3.1 Vérification et Nettoyage

La vérification systématique du dataset a confirmé l’absence de toute anomalie :

- Valeurs manquantes : **0**
- Valeurs infinies : **0**
- Valeurs hors intervalle $[0, 1]$: **0**

Les données sont prêtes pour la modélisation sans nécessiter de nettoyage supplémentaire.

3.2 Stratégie de Normalisation

Trois stratégies de normalisation ont été comparées :

TABLE 4 – Comparaison des stratégies de normalisation

Stratégie	Intervalle	Avantage	Retenue ?
Original $[0, 1]$	$[0, 1]$	Sans transformation, pas de fuite	Oui
StandardScaler	$\approx [-1, 3]$	Centré-réduit	Non
RobustScaler	$\approx [-1, 3]$	Robuste aux outliers	Non

Décision : Les données originales normalisées dans $[0, 1]$ sont conservées. Cette décision évite tout risque de fuite de données (*data leakage*) et préserve la nature intrinsèque des signaux.

3.3 Encodage des Labels

Les labels entiers sont transformés en vecteurs *one-hot* pour les modèles de Deep Learning (ANN, CNN, LSTM), tandis que les labels entiers sont conservés pour le Random Forest.

3.4 Séparation Train / Validation / Test

Une séparation **stratifiée** est appliquée pour préserver la distribution des classes dans chaque sous-ensemble :

TABLE 5 – Répartition des données

Ensemble	Échantillons	Proportion
Train	70 043	80% (du train original)
Validation	17 511	20% (du train original)
Test	21 892	fixé par Kaggle

La stratification garantit que chaque ensemble respecte la même distribution de classes que le dataset complet, en particulier pour les classes minoritaires.

3.5 Reformatage pour les Modèles

- **Format 2D** ($n, 187$) : pour le Random Forest
- **Format 3D** ($n, 187, 1$) : pour les modèles CNN et LSTM (ajout d’une dimension de canal)

4 Architectures Proposées

4.1 Modèle Baseline – Random Forest

Le Random Forest est choisi comme modèle de référence (*baseline*) pour plusieurs raisons :

- Robustesse au bruit et aux données déséquilibrées (`class_weight='balanced'`)
- Pas de normalisation requise
- Interprétable via l’importance des features

Configuration : 200 arbres, `class_weight='balanced'`, parallélisation complète (`n_jobs=-1`).

Validation croisée : 5-fold stratifiée sur un sous-échantillon de 20 000 exemples (F1-Macro moyen : $0,8367 \pm 0,0153$).

4.2 ANN – Réseau de Neurones Dense (MLP)

TABLE 6 – Architecture ANN (MLP)

Couche	Type	Paramètres
1	Dense(256) + BN + Dropout(0.3)	48 384
2	Dense(128) + BN + Dropout(0.3)	33 152
3	Dense(64) + BN + Dropout(0.2)	8 448
4	Dense(32) + Dropout(0.2)	2 080
5	Dense(5, softmax)	165
Total		92 229

Justification : L’ANN traite le signal ECG comme un vecteur de 187 features indépendantes. Il constitue la transition naturelle entre le ML classique et les architectures plus profondes.

Entraînement : Optimiseur Adam ($lr = 10^{-3}$), loss categorical cross-entropy, EarlyStopping (patience=10), ReduceLROnPlateau (factor=0.5, patience=5).

4.3 CNN – Réseau Convolutif (Conv1D)

TABLE 7 – Architecture CNN (Conv1D)

Couche	Configuration	Paramètres
1	Conv1D(64, k=5) + BN + MaxPool(2) + Dropout(0.2)	640
2	Conv1D(128, k=5) + BN + MaxPool(2) + Dropout(0.2)	41 088
3	Conv1D(256, k=3) + BN + GlobalAvgPool + Dropout(0.3)	98 560
4	Dense(128) + BN + Dropout(0.3)	32 896
5	Dense(5, softmax)	645
Total		174 725

Justification : Les noyaux de convolution 1D capturent les **patterns morphologiques locaux** du signal ECG (complexe QRS, ondes P et T), qui sont les caractéristiques discriminantes entre les classes. Le GlobalAveragePooling réduit le sur-apprentissage par rapport au Flatten classique.

4.4 RNN – LSTM Bidirectionnel (BiLSTM)

TABLE 8 – Architecture BiLSTM

Couche	Configuration	Paramètres
1	BiLSTM(64, return_seq=True) + BN + Dropout(0.3)	34 304
2	BiLSTM(32) + BN + Dropout(0.3)	41 472
3	Dense(64) + BN + Dropout(0.2)	4 224
4	Dense(5, softmax)	325
Total		80 325

Justification : Le LSTM est conçu pour capturer les **dépendances temporelles longues**. L’aspect bidirectionnel permet de lire le signal ECG dans les deux sens, enrichissant la représentation temporelle.

5 Expérimentations et Résultats

5.1 Métriques de Performance Globales

TABLE 9 – Comparaison globale des modèles sur le jeu de test

Modèle	Accuracy	F1-Macro	Temps (s)	Paramètres
Random Forest	97,20%	0,8554	79	–
ANN (MLP)	98,11%	0,8964	60	92 229
LSTM (BiLSTM)	98,03%	0,8944	719	80 325
CNN (Conv1D)	98,67%	0,9192	190	174 725

Observations :

- Le CNN est le **meilleur modèle** sur toutes les métriques
- Le LSTM est le plus lent ($\times 12$ par rapport à l’ANN) pour des performances inférieures au CNN
- Tous les modèles DL surpassent la baseline Random Forest

5.2 Résultats par Classe

TABLE 10 – F1-Score par classe pour chaque modèle

Classe	RF	ANN	LSTM	CNN	Difficulté
Normal (0)	0,98	0,99	0,99	0,99	Facile
Supraventriculaire (1)	0,72	0,79	0,79	0,81	Difficile
Ventriculaire (2)	0,92	0,95	0,95	0,97	Moyenne
Fusion (3)	0,69	0,80	0,79	0,79	Très difficile
Inconnu (4)	0,97	0,99	0,98	0,99	Facile

5.3 Analyse des Matrices de Confusion

L'analyse des matrices de confusion révèle un patron d'erreur commun à tous les modèles : **les erreurs convergent systématiquement vers la classe Normal**.

TABLE 11 – Nombre de battements mal classés comme "Normal" par modèle

Classe réelle	RF	ANN	LSTM	CNN
Supraventriculaire → Normal	238	153	141	137
Ventriculaire → Normal	172	46	72	29
Fusion → Normal	54	27	34	24
Inconnu → Normal	89	18	32	10

Le CNN réduit systématiquement les erreurs vers Normal sur toutes les classes, confirmant sa supériorité dans la capture des patterns discriminants.

5.4 Courbes d'Apprentissage

L'analyse des courbes de validation révèle :

- **ANN** : Convergence régulière et stable sur 50 époques. Pas d'overfitting (train loss $\approx$ val loss). ReduceLROnPlateau déclenché 2 fois.
- **CNN** : Quelques instabilités en début d'entraînement (époques 1-2) dues au warmup GPU CUDA. Convergence stable à partir de l'époque 16. Meilleure val loss finale (0,0449 à l'époque 40).
- **LSTM** : Oscillations de la val loss aux époques 17-26, typiques des réseaux récurrents sur séquences courtes. Stabilisation après réduction du learning rate (époque 35).

6 Discussion

6.1 Pourquoi le CNN surpasse-t-il le LSTM ?

Cette observation peut sembler contre-intuitive pour des séries temporelles, mais s'explique par la nature des signaux ECG :

- Les signaux ECG ont **187 points** – c'est une séquence relativement **courte**
- Les patterns discriminants (complexe QRS, ondes P et T) sont des structures **locales** sur quelques dizaines de points
- Le CNN capture ces patterns via ses noyaux de convolution (taille 3 à 5)
- Le LSTM est optimisé pour les **longues dépendances temporelles**, moins présentes dans ce contexte

6.2 Analyse des Erreurs

Les deux classes les plus difficiles sont :

- **Supraventriculaire (2,54% des données)** : Recall maximal de 72% avec le CNN. Ces battements ont une morphologie similaire aux battements normaux, ce qui explique les confusions fréquentes. En contexte médical, cette erreur est **critique** car elle correspond à une arythmie non détectée.
- **Fusion (0,73% des données)** : La Fusion est par définition un battement hybride entre Normal et Ventriculaire. Sa morphologie mixte rend sa classification intrinsèquement difficile, même pour un cardiologue.

6.3 Impact du Déséquilibre de Classes

Le déséquilibre (Normal = 82,77%) est la principale limite de tous les modèles. Malgré l'utilisation de `class_weight='balanced'` pour le Random Forest et de la stratification pour le split, les classes minoritaires restent sous-représentées dans les processus d'apprentissage des réseaux de neurones.

6.4 Limites des Modèles

- **Généralisation** : Les modèles sont entraînés sur MIT-BIH. Leurs performances sur d'autres bases ECG (matériel différent, population différente) restent à valider.
- **Surconfiance** : Une accuracy de 98,67% peut masquer des faiblesses sur les classes minoritaires cliniquement importantes.
- **Interprétabilité** : Les modèles DL (CNN, LSTM) sont des boîtes noires. En contexte médical, l'explicabilité est essentielle (XAI, Grad-CAM).

7 Conclusion

Ce projet a permis de mettre en œuvre un pipeline complet de classification de signaux ECG, depuis l’analyse exploratoire jusqu’à la comparaison de quatre modèles distincts.

Le CNN Conv1D est le modèle le plus performant, atteignant une accuracy de 98,67% et un F1-Macro de 0,9192 sur le jeu de test, grâce à sa capacité à capturer les patterns morphologiques locaux des signaux ECG (complexe QRS, ondes P et T).

Les principaux enseignements de ce projet sont :

1. **Le Deep Learning surpasse le ML classique** : gain de +6,38% en F1-Macro (CNN vs Random Forest)
2. **CNN > LSTM pour les séquences courtes** : les patterns locaux priment sur les dépendances temporelles longues
3. **Le déséquilibre de classes est le principal défi** : les classes minoritaires (Supraventriculaire, Fusion) restent sous-classifiées
4. **La régularisation est essentielle** : BatchNorm + Dropout + EarlyStopping ont prévenu tout sur-apprentissage

Perspectives d’amélioration :

- Gestion du déséquilibre via SMOTE ou `class_weight` pour les réseaux DL
- Architecture hybride CNN + LSTM pour combiner patterns locaux et temporels
- Optimisation des hyperparamètres (Keras Tuner, Optuna)
- Techniques d’explicabilité (Grad-CAM 1D) pour la confiance clinique

7 Références

- [1] Fazeli, S. (2017). *ECG Heartbeat Categorization Dataset*. Kaggle. <https://www.kaggle.com/datasets/shayanfazeli/heartbeat>
- [2] Moody, G.B., Mark, R.G. (2001). *The impact of the MIT-BIH Arrhythmia Database*. IEEE Engineering in Medicine and Biology Magazine, 20(3), 45–50.
- [3] Kachuee, M., Fazeli, S., Sarrafzadeh, M. (2018). *ECG Heartbeat Classification : A Deep Transferable Representation*. IEEE International Conference on Healthcare Informatics (ICHI).
- [4] LeCun, Y., Bottou, L., Bengio, Y., Haffner, P. (1998). *Gradient-based learning applied to document recognition*. Proceedings of the IEEE, 86(11), 2278–2324.
- [5] Hochreiter, S., Schmidhuber, J. (1997). *Long Short-Term Memory*. Neural Computation, 9(8), 1735–1780.
- [6] Breiman, L. (2001). *Random Forests*. Machine Learning, 45(1), 5–32.
- [7] Chollet, F. et al. (2021). *Keras Documentation*. <https://keras.io>